COSC2637/2633 Big Data Processing
Week 7: Tutorial/Lab class

Objectives

- Create, access, and use an AWS EMR cluster to run Hive to access a public dataset, and run pig scripts

Task 1 – Run Apache Pig in Batch mode (script) by writing the Pig Latin script in a single file with .pig extension

Upload taxi-one-day-dataset.csv to HDFS (if forget how to do that, visit week 3 lab materials). Suppose the HDFS path is **/user/xxx/taxi-one-day-dataset.csv**.

- **Write a Pig script and name it, for example, sample.pig**

```
taxi = LOAD 'hdfs:///user/ke/taxi-one-day-dataset.csv'
       USING PigStorage(',')
       as (pickupTime:chararray, dropoff_datetime:chararray,
           passenger_count:int, trip_distance:double,
           pickupLocation:int, dropoffLocation:int, fare:double);
taxi_fare_order = ORDER taxi BY fare DESC;
trip_limit = LIMIT taxi_fare_order 10;
Dump trip_limit;
```

Note: the DUMP operator will display the results to your terminal screen. Instead of Dump, you can try to store the output using

```
STORE trip_limit INTO 'hdfs:///user/xxx/tutorial-results';
```

- **Runn the script in MapReduce mode**

```
$ pig -x mapreduce sample.pig
```

The result will look like

```
2026-08-31 22:14:37,508 INFO util.MapRedUtil: Total input paths to process : 1
(26:18.0,34:29.0,1,4.29,68,265,389.12)
(46:26.0,03:38.0,3,59.68,264,264,385.3)
(55:20.0,55:28.0,3,0.0,265,265,332.54)
(11:03.0,06:55.0,1,48.9,164,265,314.1)
(40:41.0,12:27.0,1,70.7,138,265,276.06)
(53:09.0,38:57.0,1,32.76,216,1,263.82)
(32:36.0,43:18.0,1,4.52,265,265,261.04)
(49:17.0,31:01.0,2,64.63,158,265,250.38)
(22:20.0,23:39.0,1,0.0,234,234,230.3)
(13:24.0,13:26.0,1,0.0,265,265,226.0)
85342 [main] INFO  org.apache.pig.Main  - Pig script completed in 1 minute, 25 s
econds and 383 milliseconds (85383 ms)
2026-08-31 22:14:37,592 INFO pig.Main: Pig script completed in 1 minute, 25 seco
nds and 383 milliseconds (85383 ms)
[hadoop@ip-192-168-24-87 ~]$
```

- **Pig in Interactive Mode (Grunt shell)**

  ```
  $ pig -x mapreduce
  ```

  Either of these commands gives you the Grunt shell prompt as shown below.

  ```
  grunt>_
  ```

  Then, run each line in sample.pig again.

```
[hadoop@ip-192-168-19-79 ~]$ pig -x mapreduce
19/08/10 10:11:11 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
19/08/10 10:11:11 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
19/08/10 10:11:11 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
19/08/10 10:11:12 INFO pig.Main: Loaded log4j properties from file: /etc/pig/co
230  [main] INFO  org.apache.pig.Main  - Apache Pig version 0.17.0 (r: unknown)
19/08/10 10:11:12 INFO pig.Main: Apache Pig version 0.17.0 (r: unknown) compile
236  [main] INFO  org.apache.pig.Main  - Logging error messages to: /mnt/var/lo
19/08/10 10:11:12 INFO pig.Main: Logging error messages to: /mnt/var/log/pig/pi
354  [main] INFO  org.apache.pig.impl.util.Utils  - Default bootup file /home/h
19/08/10 10:11:12 INFO util.Utils: Default bootup file /home/hadoop/.pigbootup
19/08/10 10:11:15 INFO Configuration.deprecation: mapred.job.tracker is depreca
3088 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngin
:8020
19/08/10 10:11:15 INFO executionengine.HExecutionEngine: Connecting to hadoop f
5799 [main] INFO  org.apache.pig.PigServer  - Pig Script ID for the session: PI
19/08/10 10:11:17 INFO pig.PigServer: Pig Script ID for the session: PIG-defaul
19/08/10 10:11:20 INFO impl.TimelineClientImpl: Timeline service address: http:
8256 [main] INFO  org.apache.pig.backend.hadoop.PigATSClient  - Created ATS Hoo
19/08/10 10:11:20 INFO hadoop.PigATSClient: Created ATS Hook
grunt>
```

(using “grunt>quit” to exit grunt-shell)

What other interesting pig script can you make on this data? For more resource of Pig, please visit https://pig.apache.org/docs/r0.17.0/start.html

Task 2 – Pig script with join

In the lecture slide, the dummy code of pig script has been given.

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(count,10);
8.  store topUrls into '/data/topUrls';
```

However, this pig script cannot be executed directly even though the logic is correct. The following is the solution. Source code join_task1.pig and input data files can be found next to lab materials).

```
visits = load 'hdfs:///visits' using PigStorage(',') as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
urlInfo = load 'hdfs:///urlInfo' using PigStorage(',') as (url, category, pRank);
joinInfo = join visitCounts by url, urlInfo by url;
gCategories = group joinInfo by category;
topUrls = foreach gCategories generate TOP(2,1,joinInfo);
store topUrls into 'hdfs:///topUrls';
```

```
[hadoop@ip-192-168-24-243 ~]$ pig -x mapreduce join_task1.pig
```

If running successfully, you can see output **topUrls** in the specified folder

```
[hadoop@ip-192-168-20-159 ~]$ hadoop fs -ls /
Found 7 items
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-09-02 05:38 /apps
drwxrwxrwt   - hdfs   hdfsadmingroup          0 2023-09-02 06:12 /tmp
drwxr-xr-x   - hadoop hdfsadmingroup          0 2023-09-02 06:12 /topUrls
-rw-r--r--   1 hadoop hdfsadmingroup        123 2023-09-02 06:08 /urlInfo
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-09-02 05:38 /user
```

The content of **topUrls** can be viewed:

```
[hadoop@ip-192-168-20-159 ~]$ hadoop fs -cat /topUrls/*
{(abc.com,1,abc.com,News,0.5),(cnn.com,5,cnn.com,News,0.6)}
{(youtube.com,1,youtube.com,Music,0.9),(spotify.com,4,spotify.com,Music,0.4)}
{(flikr.com,5,flikr.com,Photos,0.8)}
```

Another example is join_task2.pig which can also be found next to lab materials

```
visits = load 'hdfs:///visits' using PigStorage(',') as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
urlInfo = load 'hdfs:///urlInfo' using PigStorage(',') as (url, category, pRank);
joinInfo = join visitCounts by url, urlInfo by url;
joinInfoOutput = foreach joinInfo generate visitCounts::url, urlInfo::category, urlInfo::pRa
store joinInfoOutput into 'hdfs:///output';
```

```
[hadoop@ip-192-168-24-243 ~]$ pig -x mapreduce join_task2.pig
```

Note **::** is disambiguate operator. After JOIN, COGROUP, CROSS, or FLATTEN operations, the field names have the original alias and the disambiguate operator :: prepended in the schema. The disambiguate operator is used to identify field names in case there is an ambiguity. You can learn more from https://pig.apache.org/docs/latest/basic.html#disambiguate

You can also run pig scripts in local mode.

```
$ pig -x local join_task1.pig
$ pig -x local join_task2.pig
```

Make sure the file path is correctly specified in the script. For example, change 'hdfs:///user/ke/visits' to ‘visits’ if the file “visits” is in the same folder

- **How to develop/debug a pig script in MapReduce platform**
  1. Go to the Grunt shell from the master node of the AWS EMR cluster

     $ pig -x mapreduce

     grount >
  2. Run each line of the script. For example

```
grunt> visits = load 'hdfs:///visits' using PigStorage(',') as (user, url, time);
23/08/28 10:21:22 INFO Configuration.deprecation: yarn.resourcemanager.system-metrics
-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
grunt>
```

To see whether the visits are what you expected, you can dump visits.

```
grunt> visits = load 'hdfs:///visits' using PigStorage(',') as (user, url, time);
23/08/28 10:21:22 INFO Configuration.deprecation: yarn.resourcemanager.system-metrics
-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
grunt> dump visits
```

If run successfully, you will see

```
Total input paths to process : 1
23/08/28 10:26:04 INFO util.MapRedUtil: To
(Amy,cnn.com,8:00)
(Amy,spotify.com,8:00)
(John,cnn.com,9:00)
(John,google.com,1:00)
(Amy,cnn.com,11:00)
(Amy,flikr.com,8:00)
(Amy,cnn.com,8:00)
(Fred,cnn.com,12:00)
(Fred,flikr.com,8:00)
(Amy,bbc.com,4:00)
(Fred,bbc.com,6:00)
(Amy,spotify.com,8:00)
(Amy,spotify.com,3:00)
(Amy,youtube.com,12:00)
(Amy,spotify.com,9:00)
(Fred,flikr.com,9:00)
(Fred,abc.com,5:00)
(Fred,flikr.com,8:00)
(Fred,flikr.com,8:00)
grunt>
```

You can also learn the schema of visits using the *describe* command

```
grunt> describe visits
visits: {user: bytearray,url: bytearray,time: bytearray}
```

or learn the content of visits using the *illustrate* command

```
grunt> illustrate visits
```

```
-------------------------------------------------------------------------------
| visits     | user:bytearray    | url:bytearray    | time:bytearray    |
-------------------------------------------------------------------------------
|            | Amy               | cnn.com          | 8:00              |
-------------------------------------------------------------------------------
```

3. After that, run the next line until all lines run correctly. If one line cannot run successfully or the output is not what you expected, you need to debug for this line. The following two sites are useful if you have questions like how to use ToDate function to handle time, how to use group, and how to use foreach to generate the expected fields in the output.

   https://pig.apache.org/docs/latest/basic.html

   https://www.tutorialspoint.com/apache_pig/apache_pig_group_operator.htm

4. Then, run the script

```
[hadoop@ip-192-168-24-243 ~]$ pig -x mapreduce join_task1.pig
```

```
[hadoop@ip-192-168-24-243 ~]$ pig -x mapreduce join_task2.pig
```

If you have developed a UDF, you need to register the UDF as the first line of the script.

Task 3 – Hive Basics

- **Enter Hive shell**

```
[hadoop@ip-192-168-30-41 ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.
properties Async: false
hive>
```

- **Create Database in Hive**

```
hive> show databases;
OK
default
Time taken: 1.379 seconds, Fetched: 1 row(s)
hive> create database myDataBase;
OK
Time taken: 0.532 seconds
hive> show databases
    > ;
OK
default
mydatabase
Time taken: 0.044 seconds, Fetched: 2 row(s)
hive>
```

- **Drop a Database in Hive**

```
hive> drop database mydatabase
    > ;
OK
Time taken: 0.187 seconds
hive> show databases
    > ;
OK
default
Time taken: 0.038 seconds, Fetched: 1 row(s)
```

- **Hive - create and alter a table in mydatabase**

```
hive> use mydatabase;
OK
Time taken: 0.043 seconds
hive> create table firsttable(empid int, empname string);
OK
Time taken: 0.096 seconds
hive> show tables;
OK
firsttable
Time taken: 0.043 seconds, Fetched: 1 row(s)
hive> Alter table firsttable RENAME to secondtable;
OK
Time taken: 0.168 seconds
hive> show tables;
OK
secondtable
Time taken: 0.056 seconds, Fetched: 1 row(s)
```

- **Drop a Database**

```
hive> drop table secondtable;
OK
Time taken: 0.291 seconds
hive> show tables;
OK
Time taken: 0.031 seconds
```

- **Hive – create table and load data to the created table (internal table only).**

Hive deals with two types of table structures like Internal and External tables depending on the loading and design of schema in Hive

– Hive Internal Table

It is the default table in Hive. When the user creates a table in Hive without specifying it as external, then by default, an internal table gets created in a specific location in HDFS. By default, an internal table will be created in a folder path similar to /user/hive/warehouse directory of HDFS. We can override the default location by the location property during table creation. If we drop the managed table or partition, the table data and the metadata associated with that table will be deleted from the HDFS.

– Hive External Table

We create an external table for external use as when we want to use the data outside the Hive. External tables are stored outside the warehouse directory. They can access data stored in sources such as remote HDFS locations or Azure Storage Volumes. Whenever we drop the external table, then only the metadata associated with the table will get deleted, the table data remains untouched by Hive.

```
hive>create table if not exists mydatabase.taxi (
    pickup_datetime string,
    dropoff_datetime string,
    passenger_count double,
    trip_distance double,
    pulocationid int,
    dolocationid int,
    total_amount double)
    COMMENT 'Taxi trip information'
    Row format delimited
    Fields Terminated By ','
    Lines Terminated By '\n'
    Stored AS TextFile;
```

```
hive> create table if not exists mydatabase.taxi (
    > pickup_datetime string,
    > dropoff_datetime string,
    > passenger_count double,
    > trip_distance double,
    > pulocationid int,
    > dolocationid int,
    > total_amount double)
    > COMMENT 'Taxi trip information'
    > Row format delimited
    > Fields Terminated By ','
    > Lines Terminated By '\n'
    > Stored AS TextFile;
OK
Time taken: 0.041 seconds
```

```
hive> use mydatabase;
OK
Time taken: 0.038 seconds
hive> show tables;
OK
taxi
Time taken: 0.044 seconds, Fetched: 1 row(s)
```

In HDFS, the table can be found in /user/hive/warehouse/mydatabase.db/

```
[hadoop@ip-192-168-24-243 ~]$ hadoop fs -ls /user/hive/warehouse/
Found 1 items
drwxrwxrwt   - hadoop hdfsadmingroup          0 2025-09-07 03:34 /user/hive/warehouse/mydatabase.db
```

```
[hadoop@ip-192-168-24-243 ~]$ hadoop fs -ls /user/hive/warehouse/mydatabase.db
Found 1 items
drwxrwxrwt   - hadoop hdfsadmingroup          0 2025-09-07 03:34 /user/hive/warehouse/mydatabase.db/taxi
```

If you want to delete the table, just

```
hive>drop table taxi
```

- **HIVE – Load data in file "/user/e20925/taxi-one-day-dataset.csv" on HDFS to the new table.**

```
hive>LOAD DATA INPATH '/user/e20925/taxi-one-day-dataset.csv' INTO table taxi;
```

```
hive> LOAD DATA INPATH '/user/e20925/taxi-one-day-dataset.csv' INTO table taxi;
Loading data to table mydatabase.taxi
OK
Time taken: 1.225 seconds
```

Note after loading, the file "/user/e20925/taxi-one-day-dataset.csv" is deleted from HDFS. Now, you can find the file "taxi-one-day-dataset.csv" moved to folder for HIVE "/user/hive/warehouse/mydatabase.db/taxi". You can try to view the content of the file as follows:

```
[hadoop@ip-192-168-24-243 ~]$ hadoop fs -ls /user/hive/warehouse/mydatabase.db/taxi
Found 1 items
-rwxrwxrwt   1 hadoop hdfsadmingroup    3607816 2025-09-07 02:38 /user/hive/warehouse/mydatabase.db/taxi/taxi-one-day-dataset.csv
[hadoop@ip-192-168-24-243 ~]$ hadoop fs -cat /user/hive/warehouse/mydatabase.db/taxi/*
```

You cannot query content of the file directly like SQL. But you can do it via HIVE!!!

- **HIVE – a simple query on table taxi**

```
hive> select * from mydatabase.taxi where total_amount>200;
OK
07:08.0 07:38.0 1.0     55.6    265     265     216.3
11:03.0 06:55.0 1.0     48.9    164     265     314.1
22:20.0 23:39.0 1.0     0.0     234     234     230.3
32:36.0 43:18.0 1.0     4.52    265     265     261.04
26:18.0 34:29.0 1.0     4.29    68      265     389.12
13:24.0 13:26.0 1.0     0.0     265     265     226.0
16:01.0 36:46.0 1.0     13.81   132     265     200.3
53:09.0 38:57.0 1.0     32.76   216     1       263.82
49:17.0 31:01.0 2.0     64.63   158     265     250.38
55:20.0 55:28.0 3.0     0.0     265     265     332.54
59:01.0 59:21.0 1.0     31.2    138     1       202.26
46:26.0 03:38.0 3.0     59.68   264     264     385.3
40:41.0 12:27.0 1.0     70.7    138     265     276.06
Time taken: 0.177 seconds, Fetched: 13 row(s)
```

More of Hive: [Hive shell - Tutorial (vskills.in)]

- **You run Hive in command line by using option -e**

```
[hadoop@ip-192-168-18-122 ~]$ hive -e 'show databases'

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.
properties Async: false
OK
default
Time taken: 3.293 seconds, Fetched: 1 row(s)
```

```
[hadoop@ip-192-168-18-122 ~]$ hive -e 'create database myDatabase'

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.
properties Async: false
OK
Time taken: 4.428 seconds
[hadoop@ip-192-168-18-122 ~]$ hive -e 'show databases'

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.
properties Async: false
OK
default
mydatabase
Time taken: 5.217 seconds, Fetched: 2 row(s)
```

Two tables are created in “default” database with option -S and without option -S
(-S means silent mode).

```
[hadoop@ip-192-168-18-122 ~]$ hive -S -e 'create table firsttable(empid int, empn
ame string)'
[hadoop@ip-192-168-18-122 ~]$ hive -e 'create table firsttable1(empid int, empnam
e string)'

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.p
roperties Async: false
OK
Time taken: 2.709 seconds
[hadoop@ip-192-168-18-122 ~]$ hive -S -e 'show tables'
firsttable
firsttable1
```

You can create a hive script file, for example, hive-script.sql. The hive-script.sql will create a new table named “taxi” in “mydatabase”, and load data from “taxi-one-day-dataset.csv” to the new table. Note '/taxi-one-day-dataset.csv' is the path and file name on HDFS.

```
create table if not exists mydatabase.taxi (
pickup_datetime string,
dropoff_datetime string,
passenger_count double,
trip_distance double,
pulocationid int,
dolocationid int,
total_amount double)
COMMENT 'Taxi trip information'
Row format delimited
Fields Terminated By ','
Lines Terminated By '\n'
Stored AS TextFile;

LOAD DATA INPATH '/taxi-one-day-dataset.csv' INTO table mydatabase.taxi;
```

- **Execute hive script “hive-script.sql” using option -f**

```
[hadoop@ip-192-168-18-122 ~]$ hive -f hive-script.sql

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.p
roperties Async: false
OK
Time taken: 3.038 seconds
Loading data to table mydatabase.taxi
OK
Time taken: 0.8 seconds
```

Note the file /taxi-one-day-dataset.scv is removed from path / after executing
“LOAD DATA INPATH '/taxi-one-day-dataset.csv' INTO table mydatabase.taxi;”
You can query the table mydatabase.taxi, and output the query results into a file, a.txt.

```
[hadoop@ip-192-168-18-122 ~]$ hive -S -e 'select * from mydatabase.taxi where tot
al_amount>200' > a.txt
[hadoop@ip-192-168-18-122 ~]$ more a.txt
07:08.0 07:38.0 1.0     55.6    265     265     216.3
11:03.0 06:55.0 1.0     48.9    164     265     314.1
22:20.0 23:39.0 1.0     0.0     234     234     230.3
32:36.0 43:18.0 1.0     4.52    265     265     261.04
26:18.0 34:29.0 1.0     4.29    68      265     389.12
13:24.0 13:26.0 1.0     0.0     265     265     226.0
16:01.0 36:46.0 1.0     13.81   132     265     200.3
53:09.0 38:57.0 1.0     32.76   216     1       263.82
49:17.0 31:01.0 2.0     64.63   158     265     250.38
55:20.0 55:28.0 3.0     0.0     265     265     332.54
59:01.0 59:21.0 1.0     31.2    138     1       202.26
46:26.0 03:38.0 3.0     59.68   264     264     385.3
40:41.0 12:27.0 1.0     70.7    138     265     276.06
```

If want to learn more about HIVE, you can visit the following site.
https://www.tutorialspoint.com/hive/hive_create_table.htm#

Don’t forget to terminate your cluster at end of the lab class!

*Also, if trying out any these exercises outside of the lab class, make sure you terminate your cluster and **DO NOT** leave clusters running idle for days.*

Terminate Cluster